



A stacking ensemble classifier with handcrafted and convolutional features for wafer map pattern classification

Hyungu Kang, Seokho Kang*

Department of Industrial Engineering, Sungkyunkwan University, 2066 Seobu-ro, Jangnan-gu, Suwon 16419, Republic of Korea

ARTICLE INFO

Article history:

Received 16 January 2021

Received in revised form 11 March 2021

Accepted 20 March 2021

Available online 30 March 2021

Keywords:

Semiconductor manufacturing
Wafer map pattern classification
Manual feature extraction
Convolutional neural network
Stacking ensemble

ABSTRACT

Recently, machine learning has been effectively applied in the automation of wafer map pattern classification in semiconductor manufacturing. One conventional approach is to extract handcrafted features from a wafer map and build an off-the-shelf classifier on top of the features. Another approach is to use a convolutional neural network that operates directly on a wafer map. These two approaches have different strengths for different classes of wafer map defect patterns. In this study, we present a hybrid method that leverages the advantages of both approaches to improve the classification accuracy. First, we build two base classifiers using each of the approaches. Then, we build a stacking ensemble that combines the outputs of these base classifiers for the final prediction. The stacking ensemble classifies a wafer map by assigning a larger weight to the output of the superior base classifier with respect to each defect class. We demonstrate the effectiveness of the proposed method using real-world data from a semiconductor manufacturer.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

In the semiconductor manufacturing process, fabricated wafers are tested to evaluate their electrical properties at the die level (Mann et al., 2004). Spatial information about the defects in each wafer is represented on a two-dimensional array known as a wafer map. The defect patterns of wafer maps can be categorized into several classes based on the similarity relation. Certain defect classes imply specific problems in the manufacturing process (Hansen et al., 1997; Yuan et al., 2011). For example, problematic etching operations often cause edge-ring patterns, whereas problems with chemical-mechanical polishing operations often cause circular and scratch patterns. Therefore, wafer map pattern classification is an essential task in the investigation of root causes that have their origins in the manufacturing process.

Traditionally, wafer map pattern classification depended on visual inspection by experienced engineers who relied on their experience and intuition (Shankar and Zhong, 2005). However, manually annotating a large number of wafer maps with their defect classes is time-consuming and expensive (Baly and Hajj, 2012). Moreover, engineers who work long hours can be exposed

to visual fatigue, which poses the risk of erroneous classification (Tan and Lau, 2011). Thus, there is a clear need to automate wafer map pattern classification to improve the efficiency of this process.

Recently, machine learning (ML) has been successfully applied to automate wafer map pattern classification (Baly and Hajj, 2012). A classification model, which accepts a wafer map as the input to predict the defect class, learns from wafer maps that were manually annotated with their defect classes. As a substitute for manual annotation by engineers, the model can be used to perform an automated classification of new wafer maps with reduced time and cost.

Two main approaches have been used to build a classification model for wafer map pattern classification (Santos et al., 2019). The first approach involves manual feature extraction (MFE) which transforms a wafer map into a fixed-length vector of handcrafted features. This is followed by the construction of an off-the-shelf classifier that takes the vector as input, for which various learning algorithms can be employed. The handcrafted features must be pre-defined manually based on the expertise of engineers to express the important characteristics of wafer maps with respect to the defect patterns. The second approach is the use of a convolutional neural network (CNN), which takes the wafer map itself as input without the requirement of MFE. CNNs have shown superior performance compared with MFE-based models, particularly when trained with

* Corresponding author.

E-mail addresses: viamonoh@skku.edu (H. Kang), s.kang@skku.edu (S. Kang).

a large amount of training data (Nakazawa and Kulkarni, 2018; Ishida et al., 2019).

The MFE and CNN approaches have different strengths for different defect classes. One approach can be superior to another in terms of distinguishing a certain class. The MFE approach reflects the domain knowledge of the human experts. Thus, it would be specialized for well-studied defect classes (Wu et al., 2015). The CNN approach generally works well for defect classes in which the spatial characteristics of a wafer map are distinguishable (Wang et al., 2014). The main aim of this study is to combine the two approaches to intensify their respective superiorities.

In this study, we propose a method that hybridizes the MFE and CNN approaches to improve the classification accuracy of wafer map patterns. Given a training dataset, we first build two base classifiers using the MFE and CNN approaches. Then, we build a stacking ensemble that combines the outputs of these base classifiers to make a final prediction. To classify a wafer map, the stacking ensemble assigns a larger weight to the superior base classifier's output with respect to each defect class. Thus, the proposed method is expected to be superior to individual base classifiers and more robust to the training dataset size. To investigate the effectiveness of the proposed method, we conducted experiments using real-world data collected from a semiconductor manufacturer.

The remainder of this paper is organized as follows. In Section 2, we review related work. The proposed method is described in Section 3. In Section 4, the experimental results are presented. Finally, the conclusions and future work are presented in Section 5.

2. Related work

2.1. Wafer map pattern classification

Various attempts have been made in the existing literature to apply ML to wafer map pattern classification. Depending on the way in which the features of wafer maps are extracted, existing methods can be categorized into two main approaches: MFE and CNN. Table 1 summarizes related studies reported in the literature.

As illustrated in Fig. 1a, the MFE approach comprises two phases: extracting handcrafted features and building an off-the-shelf classifier. In the first phase, useful features are manually extracted from a wafer map based on the knowledge of domain experts. The most commonly used features in the literature are density, geometry, and radon features (Saqlain et al., 2019). Using these handcrafted features, a wafer map is transformed into a fixed-length vector. The second phase entails the construction of a classification model that takes the vector as an input to make a prediction. In this phase, various conventional learning algorithms, such as logistic regression (LR), support vector machine (SVM), decision tree (DT), random forest (RF), gradient boosting machine (GBM), feed-forward neural network (FNN), and their ensembles, can be employed. Existing studies based on this approach have focused on designing superior handcrafted features to improve classification performance (Wu et al., 2015; Yu and Lu, 2016; Fan et al., 2016; Piao et al., 2018; Saqlain et al., 2019). The effectiveness of this approach largely depends on the informativeness of the handcrafted features used, although some important inherent information in a raw wafer map may be lost during the feature extraction.

The CNN approach aims to perform automated feature learning for wafer maps in a data-driven fashion, as illustrated in Fig. 1b. A CNN is designed to handle a multi-dimensional array as an input (LeCun et al., 2015). Because a wafer map is represented as a two-dimensional array, the CNN can directly process the wafer map by extracting convolutional features without requiring MFE. Owing to its superiority, this approach has been actively applied to wafer

map pattern classification in recent years (Nakazawa and Kulkarni, 2018; Kyeong and Kim, 2018; Ishida et al., 2019; Yu et al., 2019a,b; Wang and Chen, 2020; Shim et al., 2020; Jin et al., 2020; Kang, 2020; Hsu and Chien, 2020; Tsai and Lee, 2020; Saqlain et al., 2020; Kahng and Kim, 2020). This approach generally benefits from the use of a large amount of training data to improve the classification performance.

Existing methods are based on either the MFE or CNN approach. This study aims to combine the two approaches to leverage their respective advantages to improve wafer map pattern classification.

2.2. Combination of handcrafted and convolutional features

Feature extraction methods in computer vision have advanced from the use of MFE to CNNs (Zhu et al., 2016). These two approaches have different strengths. MFE is capable of reflecting domain knowledge by designing handcrafted features that are specialized in performing specific tasks (Wu et al., 2015). A CNN automatically extracts convolutional features without human effort and is capable of extracting sophisticated features underlying the data. The classification performance in several real-world applications can be improved by appropriately combining the MFE and CNN. In this regard, two approaches have been commonly used in the literature: ensemble construction and CNN architecture modification.

The first approach involves building base classifiers using different feature extraction methods and constructing an ensemble to combine the outputs of the base classifiers. Wang et al. (2014) constructed a cascade ensemble using a CNN and an RF for mitosis detection. Lee et al. (2018) combined the outputs from a CNN and an SVM for image modality classification. Nanni et al. (2018) constructed an ensemble of multiple SVMs trained on different features for bio-image classification. Taheri and Toygar (2019) built a CNN and combined its output with an MFE system based on feature-level and score-level fusions for age estimation. Hansley et al. (2018) used a score-level fusion of a CNN and an MFE system for unconstrained ear recognition.

The second approach entails injecting MFE directly into a CNN by modifying its architecture. Ahmed and Moustafa (2016) and Kashif et al. (2016) modified the CNN architecture to enable a vector of handcrafted features to be accepted as an additional input for house price estimation and tumor cell detection, respectively. Barroso-Laguna et al. (2019) added handcrafted filters to a CNN architecture for keypoint detection.

The proposed method applies the first approach for combining the MFE and CNN for wafer map pattern classification. We set a method based on the second approach as the baseline to compare our method.

3. Proposed method

For wafer map pattern classification, we formulate a C-class classification problem as follows. Let $\mathcal{D} = \{(\mathbf{X}_i, y_i)\}_{i=1}^N$ be a training dataset, where $\mathbf{X}_i \in \{0, 1\}^{p \times q}$ represents the i th wafer map and $y_i \in \{1, \dots, C\}$ represents the corresponding class label. Each wafer map in $\mathcal{D}$ belongs to one of the C predefined defect classes, denoted by $1, \dots, C$. We train a classification model f using training dataset $\mathcal{D}$. The trained model is then used to predict the label of a query wafer map $\mathbf{X}$ as $\hat{y} = f(\mathbf{X})$.

As the classification model f , the proposed method constructs a stacking ensemble that combines the outputs of two base classifiers, which are built based on the MFE and CNN approaches. The stacking ensemble assigns a larger weight to the superior base classifier for each defect class to improve the final prediction. A schematic diagram of the proposed method is shown in Fig. 1c.

Table 1
Summary of the literature on machine learning approaches for wafer map pattern classification.

Approach	Paper	Input features	Input shape	Classifier
MFE	Wu et al. (2015)	Geometry, Radon	116	SVM
	Yu and Lu (2016)	Geometry, Radon, Gray, Texture	53	JLND-AD
	Fan et al. (2016)	Geometry, Density	17	SVM
	Piao et al. (2018)	Radon	4	Ensemble (DT)
CNN	Saqlain et al. (2019)	Geometry, Density, Radon	66	Ensemble (LR, RF, GBM, FNN)
	Nakazawa and Kulkarni (2018)	Wafer Map	(286,400)	CNN
	Kyeong and Kim (2018)	Wafer Map	(56,53)	CNN
	Ishida et al. (2019)	Wafer Map	(100,100)	CNN
	Yu et al. (2019a)	Wafer Map	(224,224)	CNN
	Yu et al. (2019b)	Wafer Map	(128,128)	CNN
	Wang and Chen (2020)	Wafer Map (Polar Mapping)	(16,10)	CNN
	Shim et al. (2020)	Wafer Map	(64,64)	CNN
	Jin et al. (2020)	Wafer Map	(256,256)	CNN, SVM
	Kang (2020)	Wafer Map	(32,32)	CNN
	Hsu and Chien (2020)	Wafer Map	(300,300)	Ensemble (CNN)
	Tsai and Lee (2020)	Wafer Map	(64,64)	CNN
	Saqlain et al. (2020)	Wafer Map	(224,224)	CNN
	Kahng and Kim (2020)	Wafer Map	(96,96)	CNN
Proposed		Geometry, Density, Radon, Wafer Map	59, (64,64)	Ensemble (FNN, CNN)

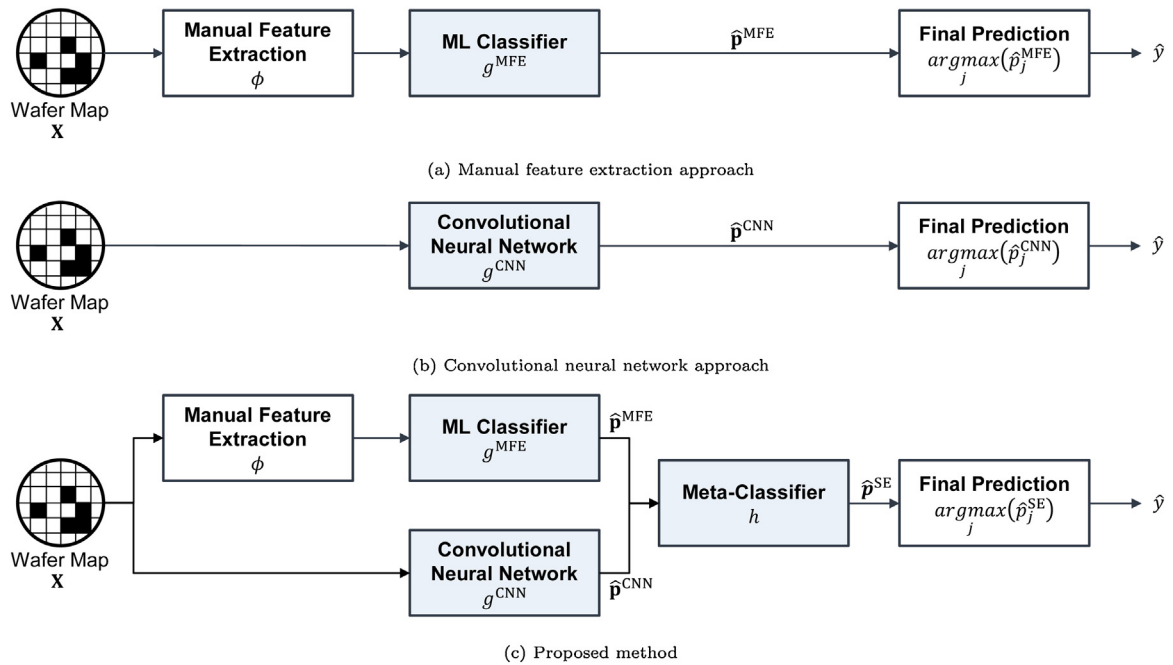


Fig. 1. Schematic diagrams of (a,b) the conventional approaches and (c) the proposed method.

Details of the training of the base classifiers and the stacking ensemble are provided in the following subsections.

3.1. Manual feature extraction-based model

We build an MFE-based model, g^{MFE} , as a base classifier. The procedure involves extracting handcrafted features from a wafer map and building the classifier on top of the features.

In the first phase, we define an MFE function ϕ that transforms a wafer map into a fixed-length vector of handcrafted features. To obtain the vector, we extract the density, geometry, and radon features, which have been widely used to describe wafer map patterns in the literature (Wu et al., 2015; Fan et al., 2016; Piao et al., 2018; Saqlain et al., 2019). For density feature extraction, the wafer map is divided into 13 areas, comprising 4 edge and 9 central areas. The defect densities of the 13 areas are used as density features; each of these is calculated by dividing the number of defective dies by the total number of dies in the area. For geometry feature

extraction, we identify multiple regions with high defect densities in the wafer map using noise filtering and designate the largest region as the salient region (Haralick and Shapiro, 1992). From the salient region, we extract the following 6 geometric features: area, perimeter, the length of major axes, the length of minor axes, solidity, and eccentricity. For radon feature extraction, the wafer map is projected onto a two-dimensional array in which the rows and columns correspond to 20 relative distances from the origin and various angles from the reference axis, respectively (Gonzalez and Woods, 2008). We calculate the mean and standard deviation for each row, resulting in a total of 40 radon features. Consequently, a total of 59 handcrafted features are obtained, including 13 density, 6 geometry, and 40 radon features. For each wafer map X , a 59-dimensional vector is obtained using the MFE function ϕ as $\phi(X)$.

In the second phase, we constitute a training dataset $\mathcal{D}^{MFE} = \{(\phi(X_i), y_i)\}_{i=1}^N$ by converting the original dataset, $\mathcal{D}$, using the MFE function ϕ . Vector $y_i \in \{0, 1\}^C$ is a one-hot representation of the corresponding class label y_i . Base classifier g^{MFE} , which uses $\phi(X)$

to predict the class label $\mathbf{y}$, is then trained using $\mathcal{D}^{\text{MFE}}$. Various learning algorithms can be used to build an off-the-shelf classifier that accepts a fixed-length vector as its input. If an FNN is used, the cost function to be minimized, J^{MFE} , could be defined based on the categorical cross-entropy loss:

$$J^{\text{MFE}} = \frac{1}{N} \sum_{i=1}^N [-\mathbf{y}_i \cdot \log g^{\text{MFE}}(\phi(\mathbf{X}_i))]. \quad (1)$$

The output of the trained classifier, $g^{\text{MFE}}(\phi(\mathbf{X}))$, is represented as a probability estimate vector $\hat{\mathbf{p}}^{\text{MFE}} \in [0, 1]^C$, where each element, $\hat{p}_j^{\text{MFE}}$, corresponds to the probability estimate of belonging to the j th class:

$$\hat{\mathbf{p}}^{\text{MFE}} = [\hat{p}_1^{\text{MFE}}, \dots, \hat{p}_C^{\text{MFE}}]^T = g^{\text{MFE}}(\phi(\mathbf{X})). \quad (2)$$

3.2. Convolutional neural network

Subsequently, we build a CNN, g^{CNN} , which is capable of directly processing a multi-dimensional array based on the notion of convolution, as another base classifier. This has demonstrated superior performance in computer vision tasks. A CNN typically comprises a stack of convolutional and fully-connected layers. In each convolutional layer, feature maps are extracted from the output of the previous layer through convolution and pooling operations. Each fully-connected layer accepts the output of the previous layer as the input to produce a feature vector. The last layer provides the probability estimates for each class, which are then used to make a final prediction. A CNN is trained in an end-to-end fashion, during which discriminative features are automatically learned by amplifying the important aspects of its input without requiring MFE. Various CNN architectures have been presented in the literature, such as VGGNet (Simonyan and Zisserman, 2014), ResNet (He et al., 2016), and DenseNet (Huang et al., 2017). These architectures are applicable to wafer map pattern classification (Nakazawa and Kulkarni, 2018; Ishida et al., 2019; Shim et al., 2020; Kang, 2020; Hsu and Chien, 2020; Tsai and Lee, 2020; Saqlain et al., 2020).

The CNN, g^{CNN} , takes wafer map $\mathbf{X}$ as input to predict the one-hot vector $\mathbf{y}$. To train the g^{CNN} , we employ the training dataset $\mathcal{D}^{\text{CNN}} = \{(\mathbf{X}_i, \mathbf{y}_i)\}_{i=1}^N$. The g^{CNN} is trained using $\mathcal{D}^{\text{CNN}}$ to minimize the cost function J^{CNN} , which is defined based on the categorical cross-entropy loss:

$$J^{\text{CNN}} = \frac{1}{N} \sum_{i=1}^N [-\mathbf{y}_i \cdot \log g^{\text{CNN}}(\mathbf{X}_i)]. \quad (3)$$

For a wafer map $\mathbf{X}$, the trained g^{CNN} produces a probability estimate vector $\hat{\mathbf{p}}^{\text{CNN}} \in [0, 1]^C$ as its output.

$$\hat{\mathbf{p}}^{\text{CNN}} = [\hat{p}_1^{\text{CNN}}, \dots, \hat{p}_C^{\text{CNN}}]^T = g^{\text{CNN}}(\mathbf{X}). \quad (4)$$

3.3. Stacking ensemble

Stacking is an ensemble method that combines outputs from heterogeneous base classifiers to improve the prediction performance. A stacking ensemble comprises base classifiers and a meta-classifier. Each base classifier is trained in a different way using a different learning algorithm to perform the target task. The meta-classifier is trained to combine the different strengths of the heterogeneous base classifiers by determining which base classifiers are more likely to be accurate for each respective class in performing the target task (Rokach, 2010). When an instance is given, the individual base classifiers classify it independently. The outputs of the base classifiers are then fed into the meta-classifier to make a final prediction (Kang et al., 2015).

To construct a stacking ensemble, any type of classifier can be employed as the meta-classifier. In this study, we choose to use multi-response linear regression (MLR) as the meta-classifier, h (Ting and Witten, 1999; Džeroski and Ženko, 2004; Kang et al., 2015). The MLR provides an intuitive interpretation of how the probability estimates from the base classifiers are combined to make a prediction. For a C -class classification problem, C linear regression models are built, each of which separates one class from the remaining classes.

The meta-classifier h takes $g^{\text{MFE}}(\phi(\mathbf{X}))$ and $g^{\text{CNN}}(\mathbf{X})$ as inputs and uses two weight matrices, $\mathbf{W}^{\text{MFE}} \in \mathbb{R}^{C \times C}$ and $\mathbf{W}^{\text{CNN}} \in \mathbb{R}^{C \times C}$, to predict $\mathbf{y}$. The training dataset is formed as $\mathcal{D}^{\text{SE}} = \{(\hat{\mathbf{p}}_i^{\text{MFE}}, \hat{\mathbf{p}}_i^{\text{CNN}}, \mathbf{y}_i)\}_{i=1}^N$, where $\hat{\mathbf{p}}_i^{\text{MFE}}$ and $\hat{\mathbf{p}}_i^{\text{CNN}}$ represent the output vectors of g^{MFE} and g^{CNN} , respectively, for $\mathbf{X}_i$. For h , we train the weight matrices $\mathbf{W}^{\text{MFE}}$ and $\mathbf{W}^{\text{CNN}}$ by minimizing the cost function J^{SE} :

$$J^{\text{SE}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{y}_i - \mathbf{W}^{\text{MFE}} \hat{\mathbf{p}}_i^{\text{MFE}} - \mathbf{W}^{\text{CNN}} \hat{\mathbf{p}}_i^{\text{CNN}})^2 + \alpha (\|\mathbf{W}^{\text{MFE}}\|^2 + \|\mathbf{W}^{\text{CNN}}\|^2), \quad (5)$$

where α denotes a hyperparameter that controls the strength of the L2 regularization for the weight matrices (Hoerl and Kennard, 1970).

Given $\hat{\mathbf{p}}^{\text{MFE}}$ and $\hat{\mathbf{p}}^{\text{CNN}}$ for wafer map $\mathbf{X}$, the trained meta-classifier h produces a probability estimate vector $\hat{\mathbf{p}}^{\text{SE}} \in [0, 1]^C$:

$$\hat{\mathbf{p}}^{\text{SE}} = [\hat{p}_1^{\text{SE}}, \dots, \hat{p}_C^{\text{SE}}]^T = h(\hat{\mathbf{p}}^{\text{MFE}}, \hat{\mathbf{p}}^{\text{CNN}}) = \mathbf{W}^{\text{MFE}} \hat{\mathbf{p}}^{\text{MFE}} + \mathbf{W}^{\text{CNN}} \hat{\mathbf{p}}^{\text{CNN}}. \quad (6)$$

To obtain $\hat{\mathbf{p}}^{\text{SE}}$, the weight matrices $\mathbf{W}^{\text{MFE}}$ and $\mathbf{W}^{\text{CNN}}$ assign larger weights to the more accurate base classifier for each class.

3.4. Prediction of new wafer maps

Once the stacking ensemble is trained, it can be used for wafer map pattern classification. When a new wafer map $\mathbf{X}_{\text{new}}$ is given, the predicted label y_{new} is obtained as follows. The wafer map $\mathbf{X}_{\text{new}}$ is first fed into two base classifiers, g^{MFE} and g^{CNN} , to produce vectors $\hat{\mathbf{p}}_{\text{new}}^{\text{MFE}}$ and $\hat{\mathbf{p}}_{\text{new}}^{\text{CNN}}$. These vectors are then used as the input for meta-classifier h to obtain the final probability estimate vector, $\hat{\mathbf{p}}_{\text{new}}^{\text{SE}}$:

$$\begin{aligned} \hat{\mathbf{p}}_{\text{new}}^{\text{SE}} &= [\hat{p}_{\text{new},1}^{\text{SE}}, \dots, \hat{p}_{\text{new},C}^{\text{SE}}]^T \\ &= h(\hat{\mathbf{p}}_{\text{new}}^{\text{MFE}}, \hat{\mathbf{p}}_{\text{new}}^{\text{CNN}}) = h(g^{\text{MFE}}(\phi(\mathbf{X}_{\text{new}})), g^{\text{CNN}}(\mathbf{X}_{\text{new}})). \end{aligned} \quad (7)$$

The class that corresponds to the largest element in $\hat{\mathbf{p}}_{\text{new}}^{\text{SE}}$ is selected as the final prediction for the defect class of $\mathbf{X}_{\text{new}}$, as shown below:

$$\hat{y}_{\text{new}} = f(\mathbf{X}_{\text{new}}) = \underset{j}{\operatorname{argmax}} \hat{p}_{\text{new},j}^{\text{SE}}. \quad (8)$$

4. Experiments

4.1. Data description

We conducted experiments using the WM-811K dataset, which was obtained from a semiconductor manufacturer (MIR Lab, 2018). This dataset comprises 811,457 wafer maps. Among them, 172,950 wafer maps were labeled with their defect classes by domain experts. We used labeled wafer maps from the dataset for the experiments. Each wafer map belonged to one of the following nine pre-defined defect classes: Center, Donut, Edge-Ring, Edge-Local, Local, Random, Near-Full, Scratch, and None. Fig. 2 presents the examples of wafer maps with different defect classes. Each defect class was associated with different specific

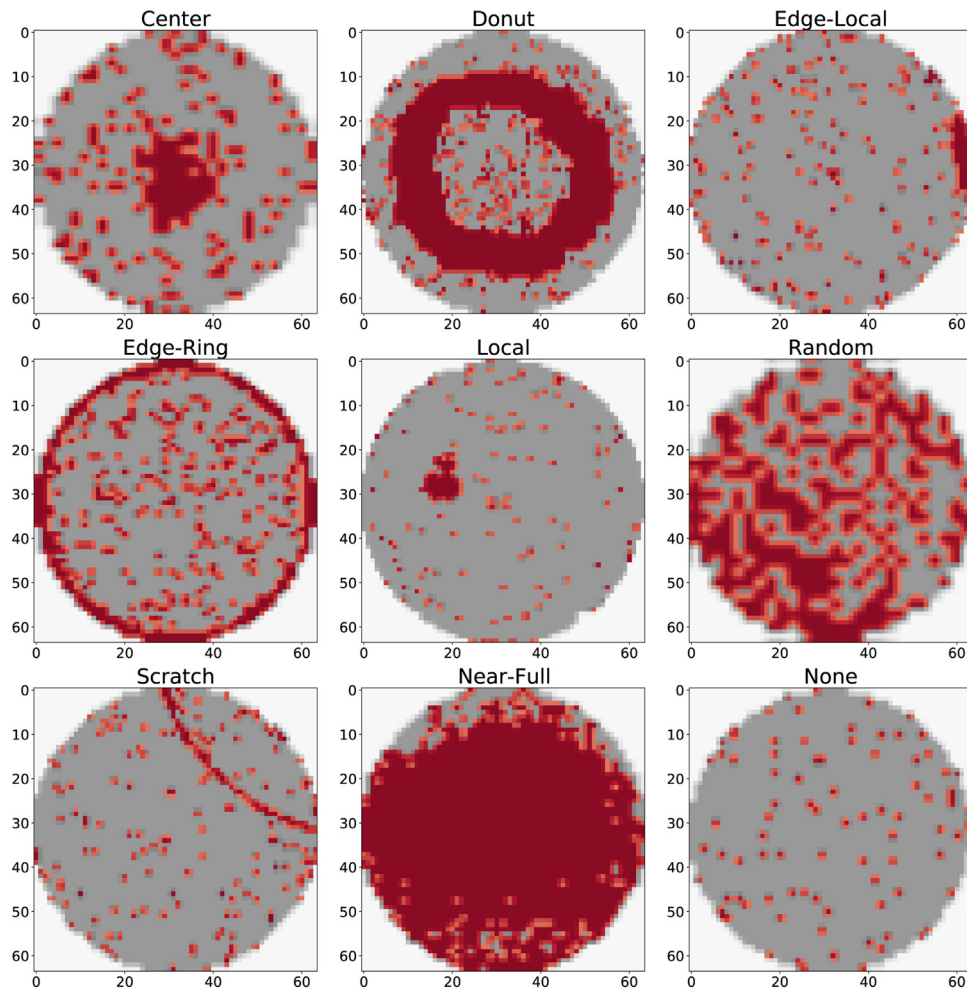


Fig. 2. Examples of wafer maps in the WM-811K dataset.

Table 2
Distribution of defect classes in the WM-811K dataset.

Class index (<i>j</i>)	Defect class	No. of wafers
1	Center	4294
2	Donut	555
3	Edge-Ring	9680
4	Edge-Local	5189
5	Local	3593
6	Random	866
7	Near-Full	149
8	Scratch	1193
9	None	147,427
Total		172,946

problems in the manufacturing process; some of these problems were abundant, whereas others rarely occurred. Accordingly, the dataset inherently exhibited an imbalanced class distribution.

We represented each wafer map as a two-dimensional array, where each element has a value of 1 if the corresponding die is defective and 0 otherwise. As the wafer map lies on a circular region of the array, the elements outside the regions were set to 0. Four abnormal wafer maps with fewer than 100 dies were removed; these were labeled as *None*. Accordingly, the dataset was reduced to 172,946 wafer maps. Table 2 presents the distribution of defect classes in the WM-811K dataset. *None* was the majority class, accounting for approximately 85.2% of the total. The shapes of the wafer maps varied from (6,21) to (300,202). To apply the

wafer map as input to the CNN, we reshaped every wafer map to (64,64).

4.2. Experimental settings

For performance evaluation, the test dataset was constructed using 10,000 wafer maps that were randomly sampled from the dataset. The remaining part was used to construct the training dataset. As the collection and labeling of wafer maps are usually expensive and time-consuming in practice, we investigated the effect of the training set size N , to evaluate the trade-off between the data acquisition cost and the classification performance. We varied N as 500, 5000, 50,000, and 162,496 by random sampling from the original training dataset. Because the distribution of defect classes was imbalanced, some minority classes may have been underrepresented when N was set to small. In the experiments, each neural network was trained using 80% of the training dataset to perform backpropagation and the remaining 20% was used as the validation part for early-stopping. For backpropagation, we used an Adam optimizer (Kingma and Ba, 2015) with a learning rate of 10^{-4} and a batch size of 32. For early-stopping, the training was terminated if the validation loss failed to decrease over 20 successive epochs or the number of epochs reached 1000.

The classification performance was evaluated in terms of the macro-average F1 ($F1_{\text{macro}}$) and micro-average F1 ($F1_{\text{micro}}$) on the test dataset (Manning et al., 2008). They are the commonly used performance metrics for wafer map pattern classification under

Table 3
FNN architecture used in this study.

Layer	Activation	Shape
Input (handcrafted features)	–	59
Fully-connected layer (128)	tanh	128
Fully-connected layer (128)	tanh	128
Fully-connected layer (9)	softmax	9

class imbalance (Yu and Lu, 2016; Ishida et al., 2019; Yu et al., 2019b; Kang, 2020; Hsu and Chien, 2020; Tsai and Lee, 2020; Saqlain et al., 2020; Kahng and Kim, 2020). $F1_{macro}$ and $F1_{micro}$ can be described as follows:

$$P_j = \frac{TP_j}{TP_j + FP_j}; \quad (9)$$

$$R_j = \frac{TP_j}{TP_j + FN_j}; \quad (10)$$

$$F1_{macro} = \frac{1}{C} \sum_{j=1}^C F1_j = \frac{1}{C} \sum_{j=1}^C \frac{2 \times P_j \times R_j}{P_j + R_j}; \quad (11)$$

$$P_{micro} = \frac{\sum_{j=1}^C TP_j}{\sum_{j=1}^C TP_j + \sum_{j=1}^C FP_j}; \quad (12)$$

$$R_{micro} = \frac{\sum_{j=1}^C TP_j}{\sum_{j=1}^C TP_j + \sum_{j=1}^C FN_j}; \quad (13)$$

$$F1_{micro} = \frac{2 \times P_{micro} \times R_{micro}}{P_{micro} + R_{micro}}, \quad (14)$$

where P and R represent the precision and recall, respectively. $F1_{macro}$ calculates the unweighted mean F1 score per class. As $F1_{macro}$ gives equal weight to each class, it can get a sense of the effectiveness on the minority classes under class imbalance. $F1_{micro}$ calculates the score globally by counting the total number of true positives, false negatives, and false positives across classes. As $F1_{micro}$ assigns equal weight to each instance, it can be dominated by the majority classes under class imbalance. $F1_{macro}$ is generally smaller than $F1_{micro}$ if the test dataset has an imbalanced class distribution.

All experiments were performed with ten independent replicates of different random seeds. We report the mean and standard deviation of each result over ten replications. The single and double asterisks (*) and **) indicate that the best model outperformed the second best at significance levels of 0.05 and 0.01, respectively, on the paired *t*-test.

4.3. Preliminary experiments

We conducted preliminary experiments to select the base classifiers for the proposed method. For the MFE-based model, we compared five off-the-shelf classifiers: LR, SVM, RF, GBM, and FNN. The 59 handcrafted features introduced in Section 3.1 were used as the input. Each feature was standardized to a mean of 0 and a standard deviation of 1 on the training dataset. We employed a small architecture for the FNN as presented in Table 3. We used the default hyperparameter settings in the scikit-learn library (Pedregosa et al., 2011) for all the models. For the CNN,

Table 4
CNN architecture used in this study.

Layer	Activation	Shape
Input (Wafer Map)	–	(64,64,3)
Convolutional layers	ReLU	(32,32,64)
- Convolution with 64 filters (3×3)		
- Convolution with 64 filters (3×3)		
- Max-Pooling (2×2)		
Convolutional layers	ReLU	(16,16,128)
- Convolution with 128 filters (3×3)		
- Convolution with 128 filters (3×3)		
- Max-Pooling (2×2)		
Convolutional layers	ReLU	(8,8,256)
- Convolution with 256 filters (3×3)		
- Convolution with 256 filters (3×3)		
- Convolution with 256 filters (3×3)		
- Max-Pooling (2×2)		
Convolutional layers	ReLU	(4,4,512)
- Convolution with 512 filters (3×3)		
- Convolution with 512 filters (3×3)		
- Convolution with 512 filters (3×3)		
- Max-Pooling (2×2)		
Convolutional layers	ReLU	(2,2,512)
- Convolution with 512 filters (3×3)		
- Convolution with 512 filters (3×3)		
- Convolution with 512 filters (3×3)		
- Max-Pooling (2×2)		
Global average pooling	–	512
Fully-connected layer (9)	softmax	9

we compared three commonly used CNN architectures: VGGNet (Simonyan and Zisserman, 2014), ResNet (He et al., 2016), and DenseNet (Huang et al., 2017). We slightly modified VGGNet by substituting the flatten operation with global average pooling and eliminating fully-connected layers, as shown in Table 4. Each CNN was pre-trained on ImageNet (Russakovsky et al., 2015).

Tables 5 and 6 compare the performance of the candidate MFE-based models and CNNs, respectively, by varying the training dataset size *N*. Among the MFE-based models, the FNN showed the best performance in most cases. Among CNNs, the VGGNet was superior to the others in most cases. Consequently, we chose the FNN and VGGNet as the base classifiers in our main experiment.

4.4. Compared models

We built the proposed model by implementing an MLR-based stacking ensemble of an MFE-based model and a CNN. We employed the FNN and VGGNet as the base classifiers in accordance with the results of the preliminary experiments. For the meta-classifier that combines the two base classifiers, we used the MLR trained with $\alpha = 0.1$.

The proposed model (Stacking-MLR) was compared with the five baseline models. Each of the two base classifiers for the stacking ensemble (MFE+FNN and CNN) was used as the baseline model. Additionally, we employed a multi-input neural network (MultiNN), that takes both a wafer map and a vector of handcrafted features as its inputs. The MultiNN had the same architecture as the CNN, except that the output vector of the global average pooling was concatenated with the vector of 59 handcrafted features. The concatenated vector was used as the input for the fully-connected layers. We also implemented stacking ensembles with different meta-classifiers (Stacking-DT and Stacking-FNN). We compared the respective effectiveness of the DT and FNN as meta-classifiers. They are nonlinear classifiers unlike MLR, and have been commonly used as meta-classifiers (Ting and Witten, 1999; Lemke et al., 2015; Caffé et al., 2012). For the DT, we used the default hyperparameter settings in the scikit-learn library (Pedregosa et al., 2011). For the

Table 5
Comparison of $F1_{\text{macro}}$ and $F1_{\text{micro}}$ for candidate MFE-based models.

Metric	Size (N)	LR	SVM	RF	GBM	FNN
$F1_{\text{macro}}$	500	0.5787 ± 0.0259*	0.3689 ± 0.0357	0.5390 ± 0.0418	0.5065 ± 0.0234	0.5558 ± 0.0376
	5000	0.7078 ± 0.0307	0.5420 ± 0.0287	0.6655 ± 0.0176	0.6527 ± 0.0279	0.6983 ± 0.0371
	50,000	0.7988 ± 0.0170	0.7636 ± 0.0166	0.7965 ± 0.0089	0.7797 ± 0.0138	0.8310 ± 0.0109**
	162,946	0.8143 ± 0.0133	0.8254 ± 0.0127	0.8345 ± 0.0126	0.8112 ± 0.0123	0.8599 ± 0.0117**
$F1_{\text{micro}}$	500	0.9401 ± 0.0033	0.9244 ± 0.0033	0.9377 ± 0.0036	0.9286 ± 0.0033	0.9415 ± 0.0044
	5000	0.9581 ± 0.0013	0.9494 ± 0.0022	0.9555 ± 0.0021	0.9514 ± 0.0019	0.9600 ± 0.0011**
	50,000	0.9657 ± 0.0017	0.9662 ± 0.0013	0.9660 ± 0.0018	0.9650 ± 0.0011	0.9712 ± 0.0010**
	162,946	0.9670 ± 0.0014	0.9718 ± 0.0010	0.9703 ± 0.0014	0.9683 ± 0.0013	0.9741 ± 0.0010**

Table 6
Comparison of $F1_{\text{macro}}$ and $F1_{\text{micro}}$ for candidate CNNs.

Metric	Size (N)	VGGNet	ResNet	DenseNet
$F1_{\text{macro}}$	500	0.4937 ± 0.0642**	0.3794 ± 0.0804	0.3868 ± 0.0494
	5000	0.6954 ± 0.0420	0.6538 ± 0.0304	0.6707 ± 0.0442
	50,000	0.8397 ± 0.0213*	0.8184 ± 0.0192	0.8252 ± 0.0129
	162,946	0.8679 ± 0.0126	0.8678 ± 0.0174	0.8775 ± 0.0182
$F1_{\text{micro}}$	500	0.9292 ± 0.0092**	0.9157 ± 0.0077	0.9170 ± 0.0053
	5000	0.9607 ± 0.0027**	0.9460 ± 0.0038	0.9504 ± 0.0053
	50,000	0.9731 ± 0.0023**	0.9677 ± 0.0030	0.9692 ± 0.0018
	162,946	0.9775 ± 0.0014*	0.9756 ± 0.0013	0.9765 ± 0.0016

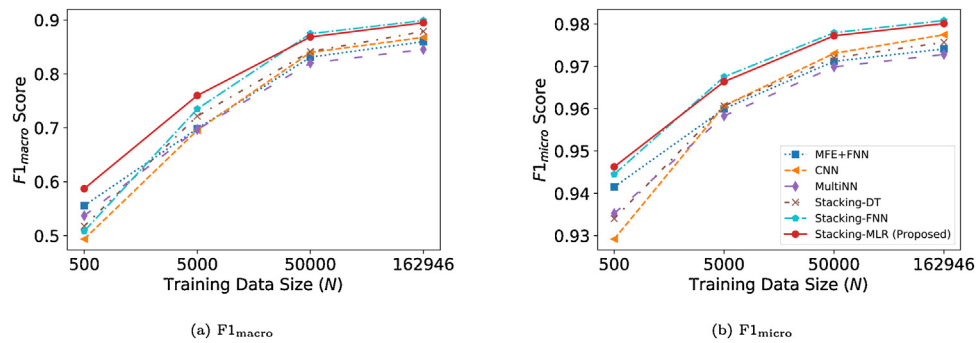


Fig. 3. Comparison of $F1_{\text{macro}}$ and $F1_{\text{micro}}$ for the baseline and proposed models.

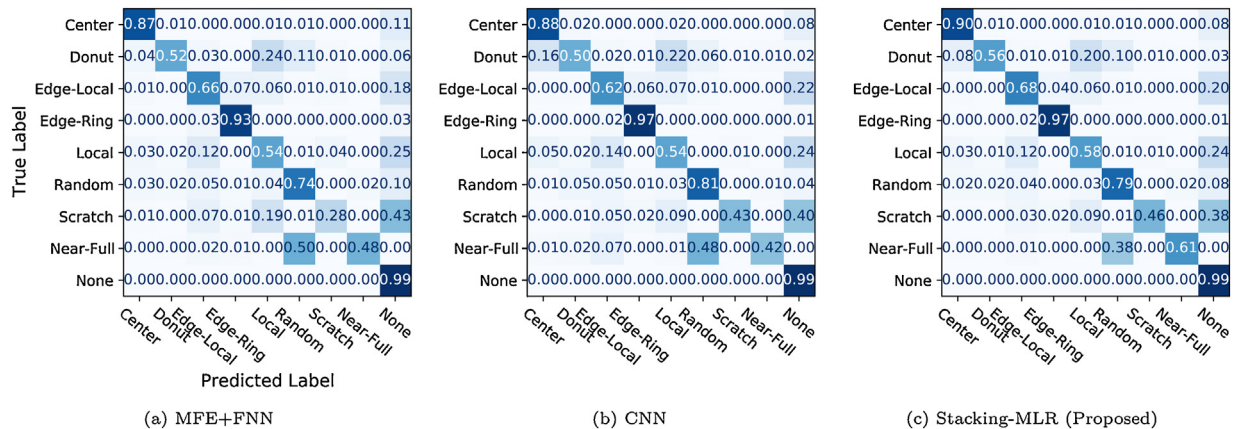


Fig. 4. Confusion matrices of the base classifiers and the proposed model at $N=5000$.

Table 7
Comparison of $F1_{\text{macro}}$ and $F1_{\text{micro}}$ for the baseline and proposed models.

Metric	Size (N)	MFE+FNN	CNN	MultiNN	Stacking-DT	Stacking-FNN	Stacking-MLR (proposed)
$F1_{\text{macro}}$	500	0.5558 ± 0.0376	0.4937 ± 0.0642	0.5371 ± 0.0902	0.5179 ± 0.0368	0.5085 ± 0.0662	0.5872 ± 0.0542*
	5000	0.6983 ± 0.0371	0.6954 ± 0.0420	0.6969 ± 0.0309	0.7217 ± 0.0220	0.7350 ± 0.0556	0.7599 ± 0.0202
	50,000	0.8310 ± 0.0109	0.8397 ± 0.0213	0.8201 ± 0.0174	0.8415 ± 0.0116	0.8746 ± 0.0088*	0.8686 ± 0.0078
	162,946	0.8599 ± 0.0117	0.8679 ± 0.0126	0.8455 ± 0.0170	0.8789 ± 0.0094	0.8991 ± 0.0096	0.8949 ± 0.0121
$F1_{\text{micro}}$	500	0.9415 ± 0.0044	0.9292 ± 0.0092	0.9352 ± 0.0084	0.9340 ± 0.0040	0.9445 ± 0.0030	0.9462 ± 0.0042
	5000	0.9600 ± 0.0011	0.9607 ± 0.0027	0.9583 ± 0.0020	0.9607 ± 0.0025	0.9675 ± 0.0024*	0.9664 ± 0.0018
	50,000	0.9712 ± 0.0010	0.9731 ± 0.0023	0.9699 ± 0.0015	0.9720 ± 0.0021	0.9779 ± 0.0011*	0.9772 ± 0.0013
	162,946	0.9741 ± 0.0010	0.9775 ± 0.0014	0.9728 ± 0.0015	0.9757 ± 0.0011	0.9808 ± 0.0010*	0.9801 ± 0.0010

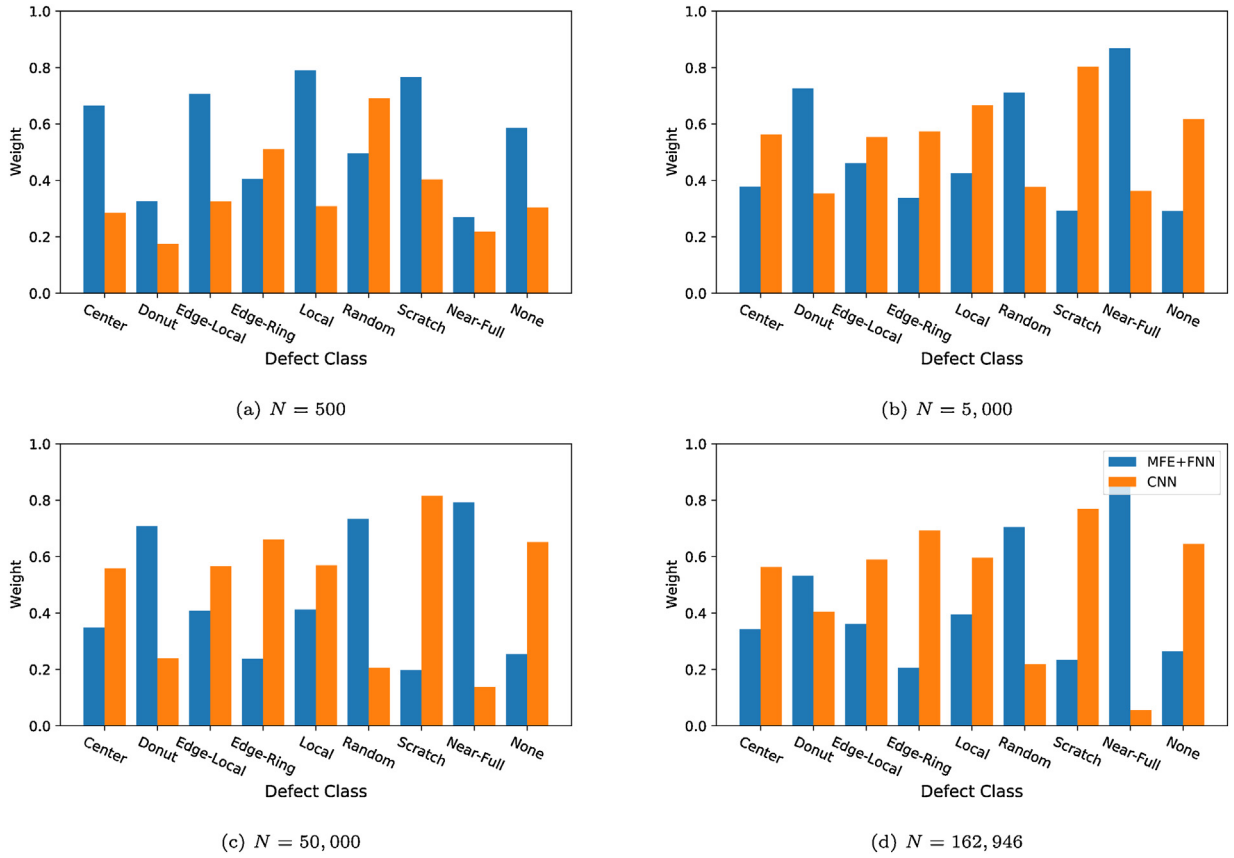


Fig. 5. Comparison of MLR weights for each base classifier and defect class.

FNN, we used a small architecture with one hidden layer and 10 hidden nodes.

In summary, six models were compared in the experiments: MFE+FNN, CNN, MultiNN, Stacking-DT, Stacking-FNN, and Stacking-MLR. They were implemented in Python using the following libraries. Handcrafted features were obtained using the scikit-image library (Van der Walt et al., 2014). All the neural networks were implemented using the TensorFlow library (Abadi et al., 2016). The DT and MLR were implemented using the scikit-learn library (Pedregosa et al., 2011).

4.5. Results and discussion

Table 7 and Fig. 3 provide a comparison of the $F1_{\text{macro}}$ and $F1_{\text{micro}}$ of the baseline and proposed models for various training dataset sizes N . As shown in Table 7, the proposed model and Stacking-FNN consistently outperformed the other models for all values of N . In particular, they showed a significant increase in $F1_{\text{macro}}$, indicating that they performed better under class imbalance. We also observed that the proposed model showed competitive performance for all values of N . When $N = 500$, the proposed model yielded the best performance with a large margin. When N was larger, the proposed model performed slightly worse than the Stacking-FNN. Among the base classifiers, the MFE+FNN exhibited good performance when N was small, whereas the CNN was relatively superior when N was large.

Table 8 provides a comparison the $F1$ scores of the six models according to the defect class for various values of N . The MFE+FNN and CNN exhibited different strengths in different defect classes. For example, the MFE+FNN classified *Near-Full* better than the CNN, whereas the CNN classified *Scratch* better than the MFE+FNN

for most values of N . The proposed model outperformed the best model between the MFE+FNN and the CNN for most defect classes. This result indicates that the proposed model appropriately utilized the output of the superior base classifier for each defect class, thereby increasing the $F1$ scores for each defect class.

Fig. 4 presents the confusion matrices of the proposed model and its base classifiers at $N = 5000$. The results show that the two base classifiers were complementary. The MFE+FNN was relatively superior in classifying the wafer maps of *Edge-Local* and *Near-Full* classes, whereas it tended to misclassify *Scratch* as *None*. The CNN performed better for *Edge-Ring* and *Scratch* classes. The proposed model successfully addressed the individual weaknesses of the MFE+FNN and CNN, thereby yielding improved overall performance for all classes.

We further investigated how the proposed model worked by examining the weights of MLR assigned to different base classifiers and defect classes. Fig. 5 presents the results in the form of bar graphs that represent the diagonal entries in the weight matrices $\mathbf{W}^{\text{MFE}}$ and $\mathbf{W}^{\text{CNN}}$. The weights for the MFE+FNN and CNN with respect to the j th class were $\mathbf{W}_{jj}^{\text{MFE}}$ and $\mathbf{W}_{jj}^{\text{CNN}}$, respectively. A larger weight for a base classifier on a defect class indicates that the classifier contributes more to the final prediction for the class. The proposed model tended to assign a larger weight to the superior base classifier for each defect class. The weights for the MFE+FNN in *Donut* and *Near-Full* classes were consistently larger than those for the CNN. For small values of N , the weights for the MFE+FNN exceeded those for the CNN for most defect classes. As N increased, the weights for the CNN became larger for some defect classes such as *Center*, *Edge-Ring*, *Scratch*. We infer that assigning appropriate

Table 8
Comparison of the F1 score per defect class for the baseline and proposed models.

Size (N)	Defect class	MFE+FNN	CNN	MultiNN	Stacking-DT	Stacking-FNN	Stacking-MLR (proposed)
500	Center	0.7917 ± 0.0317	0.6926 ± 0.0504	0.7199 ± 0.0883	0.7817 ± 0.0248	0.7805 ± 0.0294	0.7943 ± 0.0322
	Donut	0.1421 ± 0.1721	0.0960 ± 0.1579	0.1041 ± 0.1457	0.0779 ± 0.1365	0.1111 ± 0.1898	0.1427 ± 0.1886
	Edge-Local	0.5318 ± 0.0737	0.2886 ± 0.1034	0.4454 ± 0.1214	0.5088 ± 0.0641	0.5374 ± 0.0457	0.5606 ± 0.0588
	Edge-Ring	0.8839 ± 0.0172	0.9240 ± 0.0189	0.8955 ± 0.0247	0.9261 ± 0.0222	0.9280 ± 0.0130	0.9308 ± 0.0229
	Local	0.4223 ± 0.0700	0.2065 ± 0.1172	0.3608 ± 0.1314	0.3401 ± 0.0668	0.3960 ± 0.0914	0.4080 ± 0.0842
	Random	0.3958 ± 0.2245	0.4818 ± 0.1944	0.5002 ± 0.2234	0.3235 ± 0.1062	0.2415 ± 0.2944	0.5678 ± 0.1731
	Scratch	0.0889 ± 0.0754	0.0781 ± 0.0691	0.0331 ± 0.0437	0.1024 ± 0.0624	0.0148 ± 0.0358	0.0999 ± 0.0691
	Near-Full	0.2491 ± 0.3052	0.1908 ± 0.2650	0.2831 ± 0.3189	0.1457 ± 0.2539	0.1022 ± 0.2418	0.2570 ± 0.2990
	None	0.9811 ± 0.0022	0.9737 ± 0.0051	0.9746 ± 0.0046	0.9791 ± 0.0029	0.9813 ± 0.0016	0.9820 ± 0.0016*
	Center	0.8900 ± 0.0171	0.8863 ± 0.0220	0.8795 ± 0.0247	0.8962 ± 0.0174	0.9150 ± 0.0104	0.9105 ± 0.0167
5000	Donut	0.5948 ± 0.1551	0.5066 ± 0.1573	0.5829 ± 0.1582	0.5804 ± 0.1252	0.5735 ± 0.2394	0.6315 ± 0.1562*
	Edge-Local	0.7024 ± 0.0112	0.6674 ± 0.0343	0.6800 ± 0.0320	0.6883 ± 0.0203	0.7464 ± 0.0186	0.7293 ± 0.0209*
	Edge-Ring	0.9388 ± 0.0068	0.9532 ± 0.0100	0.9405 ± 0.0069	0.9574 ± 0.0069	0.9607 ± 0.0047	0.9603 ± 0.0072
	Local	0.5887 ± 0.0233	0.6003 ± 0.0441	0.5790 ± 0.0354	0.5954 ± 0.0385	0.6619 ± 0.0393	0.6456 ± 0.0359
	Random	0.7181 ± 0.0481	0.7535 ± 0.0785	0.7065 ± 0.0710	0.7499 ± 0.0701	0.7667 ± 0.0479*	0.7645 ± 0.0511
	Scratch	0.3676 ± 0.0893	0.5607 ± 0.1023	0.3269 ± 0.1147	0.5200 ± 0.0909	0.5835 ± 0.0946	0.5916 ± 0.0978
	Near-Full	0.4985 ± 0.3233	0.3442 ± 0.3177	0.5914 ± 0.1783	0.5207 ± 0.2044	0.4179 ± 0.3742	0.6185 ± 0.2155
	None	0.9863 ± 0.0006	0.9861 ± 0.0010	0.9856 ± 0.0011	0.9868 ± 0.0012	0.9890 ± 0.0009**	0.9878 ± 0.0006
	Center	0.9296 ± 0.0111	0.9249 ± 0.0134	0.9218 ± 0.0101	0.9225 ± 0.0144	0.9406 ± 0.0096*	0.9382 ± 0.0105
	Donut	0.8325 ± 0.0323	0.7603 ± 0.1336	0.7841 ± 0.0642	0.8293 ± 0.0457	0.8740 ± 0.0153	0.8568 ± 0.0349
50,000	Edge-Local	0.7818 ± 0.0132	0.7889 ± 0.0207	0.7659 ± 0.0176	0.7852 ± 0.0242	0.8277 ± 0.0132	0.8256 ± 0.0166
	Edge-Ring	0.9596 ± 0.0039	0.9724 ± 0.0081	0.9589 ± 0.0060	0.9751 ± 0.0044	0.9784 ± 0.0043	0.9775 ± 0.0046
	Local	0.7184 ± 0.0185	0.7297 ± 0.0354	0.7088 ± 0.0300	0.7241 ± 0.0211	0.7769 ± 0.0197	0.7715 ± 0.0173
	Random	0.8467 ± 0.0257	0.8534 ± 0.0363	0.8412 ± 0.0419	0.8231 ± 0.0369	0.8669 ± 0.0310	0.8680 ± 0.0263
	Scratch	0.5742 ± 0.0694	0.7250 ± 0.0377	0.5918 ± 0.0591	0.7029 ± 0.0428	0.7611 ± 0.0332*	0.7477 ± 0.0363
	Near-Full	0.8474 ± 0.0779	0.8131 ± 0.0829	0.8194 ± 0.0812	0.8225 ± 0.0773	0.8544 ± 0.0620	0.8413 ± 0.0768
	None	0.9891 ± 0.0005	0.9897 ± 0.0009	0.9888 ± 0.0006	0.9893 ± 0.0011	0.9915 ± 0.0006**	0.9910 ± 0.0006
	Center	0.9323 ± 0.0090	0.9326 ± 0.0106	0.9304 ± 0.0142	0.9279 ± 0.0074	0.9455 ± 0.0070*	0.9424 ± 0.0063
	Donut	0.8615 ± 0.0468	0.8760 ± 0.0426	0.8251 ± 0.0489	0.8694 ± 0.0356	0.8843 ± 0.0441	0.8969 ± 0.0388
	Edge-Local	0.8051 ± 0.0164	0.8301 ± 0.0143	0.7939 ± 0.0132	0.8231 ± 0.0139	0.8562 ± 0.0120*	0.8492 ± 0.0148
162,496	Edge-Ring	0.9667 ± 0.0037	0.9794 ± 0.0042	0.9661 ± 0.0045	0.9782 ± 0.0051	0.9824 ± 0.0042	0.9823 ± 0.0034
	Local	0.7406 ± 0.0116	0.7715 ± 0.0234	0.7282 ± 0.0254	0.7605 ± 0.0144	0.8059 ± 0.0189*	0.7950 ± 0.0200
	Random	0.8928 ± 0.0252	0.8653 ± 0.0311	0.8527 ± 0.0508	0.8858 ± 0.0174	0.9033 ± 0.0247	0.8979 ± 0.0262
	Scratch	0.6451 ± 0.0486	0.7737 ± 0.0405	0.6494 ± 0.0695	0.7545 ± 0.0294	0.8083 ± 0.0268*	0.7928 ± 0.0327
	Near-Full	0.9053 ± 0.0587	0.7920 ± 0.0951	0.8738 ± 0.0691	0.9201 ± 0.0665	0.9142 ± 0.0516	0.9053 ± 0.0587
	None	0.9899 ± 0.0007	0.9909 ± 0.0005	0.9895 ± 0.0005	0.9901 ± 0.0005	0.9923 ± 0.0004**	0.9918 ± 0.0002

weights to the superior base classifiers improves the classification performance of the proposed model.

5. Conclusion

In this study, we proposed a hybrid method that integrates the MFE and CNN approaches to leverage their respective strengths. The proposed method constructed a stacking ensemble that combined two base classifiers, each of which was built based on the MFE and CNN, respectively, using MLR. The ensemble assigned a larger weight to the superior base classifier for each defect class. Experimental validation using a real-world dataset demonstrated the effectiveness of the proposed method. We demonstrated that the base classifiers were complementary to each other, with different strengths for different defect classes. The proposed method yielded an improved classification performance compared to the base classifiers and other baseline methods. The proposed method was also robust with respect to the size of the training dataset.

This study verified that wafer map pattern classification could be improved using both handcrafted features containing domain knowledge and convolutional features learned from the data. The improved classification performance is expected to lead to more reliable automation of wafer map pattern classification. In the future, we plan to further improve the proposed method to better address real-world data issues in wafer map pattern classification, including class imbalance and data scarcity. In addition, we intend to adopt a combination of handcrafted and convolutional features for prediction tasks in other manufacturing fields wherein the incorporation of domain knowledge can be beneficial.

Authors' contribution

Hyungu Kang: conceptualization, methodology, software, writing – original draft. Seokho Kang: conceptualization, methodology, investigation, writing – review & editing.

Conflict of interest

None declared.

Declaration of Competing Interest

The authors report no declarations of interest.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT; Ministry of Science and ICT) (Nos. NRF-2019R1A4A1024732 and NRF-2020R1C1C1003232) and the Korea Institute for Advancement of Technology (KIAT) grant funded by the Korea government (MOTIE; Ministry of Trade Industry and Energy) (No. N0002429).

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X., 2016. *TensorFlow: large-scale machine learning on heterogeneous systems*. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 265–283.

- Ahmed, E.H., Moustafa, M., 2016. House price estimation from visual and textual features. *Proceedings of the 8th International Joint Conference on Computational Intelligence*, 62–68.
- Baly, R., Hajj, H., 2012. Wafer classification using support vector machines. *IEEE Trans. Semicond. Manuf.* 25, 373–383.
- Barroso-Laguna, A., Riba, E., Ponsa, D., Mikolajczyk, K., 2019. Key.Net. Keypoint detection by handcrafted and learned CNN filters. *Proceedings of the IEEE International Conference on Computer Vision*, 5836–5844.
- Caffé, M.I.R., Perez, P.S., Baranauskas, J.A., 2012. Evaluation of stacking on biomedical data. *J. Health Informatics* 4, 67–72.
- Džeroski, S., Ženko, B., 2004. Is combining classifiers with stacking better than selecting the best one? *Mach. Learn.* 54, 255–273.
- Fan, M., Wang, Q., van der Waal, B., 2016. Wafer defect patterns recognition based on OPTICS and multi-label classification. *Proceedings of the 2016 IEEE Advanced Information Management, Communicates, Electronic and Automation Control Conference*, 912–915.
- Gonzalez, R.C., Woods, R.E., 2008. *Digital Image Processing*, 3rd ed. Prentice Hall.
- Hansen, M.H., Nair, V.N., Friedman, D.J., 1997. Monitoring wafer map data from integrated circuit fabrication processes for spatially clustered defects. *Technometrics* 39, 241–253.
- Hansley, E.E., Segundo, M.P., Sarkar, S., 2018. Employing fusion of learned and handcrafted features for unconstrained ear recognition. *IET Biometrics* 7, 215–223.
- Haralick, R.M., Shapiro, L.G., 1992. *Computer and Robot Vision*. Addison-Wesley.
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Identity mappings in deep residual networks. *Proceedings of European Conference on Computer Vision*, 630–645.
- Hoerl, A.E., Kennard, R.W., 1970. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics* 12, 55–67.
- Hsu, C.-Y., Chien, J.-C., 2020. Ensemble convolutional neural networks with weighted majority for wafer bin map pattern classification. *J. Intell. Manuf.*
- Huang, G., Liu, Z., Van Der Maaten, L., Weinberger, K.Q., 2017. Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 4700–4708.
- Ishida, T., Nitta, I., Fukuda, D., Kanazawa, Y., 2019. Deep learning-based wafer-map failure pattern recognition framework. *Proceedings of the 20th International Symposium on Quality Electronic Design*, 291–297.
- Jin, C.H., Kim, H.-J., Piao, Y., Li, M., Piao, M., 2020. Wafer map defect pattern classification based on convolutional neural network features and error-correcting output codes. *J. Intell. Manuf.* 31, 1861–1875.
- Kahng, H., Kim, S.B., 2020. Self-supervised representation learning for wafer bin map defect pattern classification. *IEEE Trans. Semicond. Manuf.*
- Kang, S., Cho, S., Kang, P., 2015. Multi-class classification via heterogeneous ensemble of one-class classifiers. *Eng. Appl. Artif. Intell.* 43, 35–43.
- Kang, S., 2020. Rotation-invariant wafer map pattern classification with convolutional neural networks. *IEEE Access* 8, 170650–170658.
- Kashif, M.N., Raza, S.E.A., Sirinukunwattana, K., Arif, M., Rajpoot, N., 2016. Handcrafted features with convolutional neural networks for detection of tumor cells in histology images. *Proceedings of the IEEE International Symposium on Biomedical Imaging*, 1029–1032.
- Kingma, D.P., Ba, J., 2015. Adam: a method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations*.
- Kyeong, K., Kim, H., 2018. Classification of mixed-type defect patterns in wafer bin maps using convolutional neural networks. *IEEE Trans. Semicond. Manuf.* 31, 395–402.
- LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Nature* 521, 436–444.
- Lee, S.L., Zare, M.R., Muller, H., 2018. Late fusion of deep learning and handcrafted visual features for biomedical image modality classification. *IET Image Process.* 13, 382–391.
- Lemke, C., Budka, M., Gabrys, B., 2015. Metalearning: a survey of trends and technologies. *Artif. Intell. Rev.* 44, 117–130.
- Mann, W.R., Taber, F.L., Seitzer, P.W., Broz, J.J., 2004. The leading edge of production wafer probe test technology. *Proceedings of the IEEE International Test Conference*, 1168–1195.
- Manning, C.D., Schütze, H., Raghavan, P., 2008. *Introduction to Information Retrieval*. Cambridge University Press.
- MIR Lab, 2018. LSWMD Data (Accessed 12 July 2020) <https://mirlab.org/dataSet/public>.
- Nakazawa, T., Kulkarni, D.V., 2018. Wafer map defect pattern classification and image retrieval using convolutional neural network. *IEEE Trans. Semicond. Manuf.* 31, 309–314.
- Nanni, L., Brahnam, S., Ghidoni, S., Lumini, A., 2018. Bioimage classification with handcrafted and learned features. *IEEE/ACM Trans. Comput. Biol. Bioinformatics* 16, 874–885.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E., 2011. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12, 2825–2830.
- Piao, M., Jin, C.H., Lee, J.Y., Byun, J., 2018. Decision tree ensemble-based wafer map failure pattern recognition based on radon transform-based features. *IEEE Trans. Semicond. Manuf.* 31, 250–257.
- Rokach, L., 2010. Ensemble-based classifiers. *Artif. Intell. Rev.* 33, 1–39.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al., 2015. Imagenet large scale visual recognition challenge. *Int. J. Comput. Vision* 115, 211–252.
- Santos, T., Schrunner, S., Geiger, B.C., Pfeiler, O., Zernig, A., Kaestner, A., Kern, R., 2019. Feature extraction from analog wafermaps: A comparison of classical image processing and a deep generative model. *IEEE Trans. Semicond. Manuf.* 32, 190–198.
- Saqlain, M., Jargalsaikhan, B., Lee, J.Y., 2019. A voting ensemble classifier for wafer map defect patterns identification in semiconductor manufacturing. *IEEE Trans. Semicond. Manuf.* 32, 171–182.
- Saqlain, M., Abbas, Q., Lee, J.Y., 2020. A deep convolutional neural network for wafer defect identification on an imbalanced dataset in semiconductor manufacturing processes. *IEEE Trans. Semicond. Manuf.* 33, 436–444.
- Shankar, N.G., Zhong, Z.W., 2005. Defect detection on semiconductor wafer surfaces. *Microelectron. Eng.* 77, 337–346.
- Shim, J., Kang, S., Cho, S., 2020. Active learning of convolutional neural network for cost-effective wafer map pattern classification. *IEEE Trans. Semicond. Manuf.* 33, 258–266.
- Simonyan, K., Zisserman, A., 2014. Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556*.
- Taheri, S., Toygar, Ö., 2019. Multi-stage age estimation using two level fusions of handcrafted and learned features on facial images. *IET Biometrics* 8, 124–133.
- Tan, C.M., Lau, K.T., 2011. Automated wafer defect map generation for process yield improvement. *Proceedings of the 2011 International Symposium on Integrated Circuits*, 313–316.
- Ting, K.M., Witten, I.H., 1999. Issues in stacked generalization. *J. Artif. Intell. Res.* 10, 271–289.
- Tsai, T.-H., Lee, Y.-C., 2020. A light-weight neural network for wafer map classification based on data augmentation. *IEEE Trans. Semicond. Manuf.* 33, 663–672.
- Van der Walt, S., Schönberger, J.L., Nunez-Iglesias, J., Boulogne, F., Warner, J.D., Yager, N., Guillaud, E., Yu, T., 2014. scikit-image: image processing in Python. *PeerJ* 2, e453.
- Wang, R., Chen, N., 2020. Defect pattern recognition on wafers using convolutional neural networks. *Qual. Reliab. Eng. Int.* 36, 1245–1257.
- Wang, H., Cruz-Roa, A., Basavanahally, A., Gilmore, H., Shih, N., Feldman, M., Tomaszewski, J., Gonzalez, F., Madabhushi, A., 2014. Cascaded ensemble of convolutional neural networks and handcrafted features for mitosis detection. *Proceedings of the Medical Imaging 2014: Digital Pathology*, Vol. 9041, 66–75.
- Wu, M.-J., Jang, J.-S.R., Chen, J.-L., 2015. Wafer map failure pattern recognition and similarity ranking for large-scale data sets. *IEEE Trans. Semicond. Manuf.* 28, 1–12.
- Yu, J., Lu, X., 2016. Wafer map defect detection and recognition using joint local and nonlocal linear discriminant analysis. *IEEE Trans. Semicond. Manuf.* 29, 33–43.
- Yu, N., Xu, Q., Wang, H., 2019a. Wafer defect pattern recognition and analysis based on convolutional neural network. *IEEE Trans. Semicond. Manuf.* 32, 566–573.
- Yu, J., Zheng, X., Liu, J., 2019b. Stacked convolutional sparse denoising auto-encoder for identification of defect patterns in semiconductor wafer map. *Comput. Ind.* 109, 121–133.
- Yuan, T., Kuo, W., Bae, S.J., 2011. Detection of spatial defect patterns generated in semiconductor fabrication processes. *IEEE Trans. Semicond. Manuf.* 24, 392–403.
- Zhu, F., Shao, L., Xie, J., Fang, Y., 2016. From handcrafted to learned representations for human action recognition: a survey. *Image Vision Comput.* 55, 42–52.